

# Lab 2.3 — String Operations & Type Conversion

## Objective

By the end of this file, you will be able to concatenate and repeat strings, use indexing and slicing, apply common string methods (`strip`, `upper`, `capitalize`, `replace`), and convert values between `str`, `int`, and `float` using Python's built-in conversion functions.

---

## Background

### Strings in Python

A **string** (`str`) is an ordered sequence of characters enclosed in single (`' . . '`) or double (`" . . "`) quotes. Because strings are **sequences**, Python allows you to access individual characters by their position (index) and extract subsets (slices) — making text processing powerful and expressive.

### Type Conversion: Why It's Necessary

Python is **strongly typed** — it will not silently mix incompatible types. Trying to add a number to a string causes a `TypeError`. Type conversion functions (`int()`, `float()`, `str()`) let you explicitly change a value's type so operations work correctly. This is especially important when reading user input, because `input()` **always returns a string** — even if the user types a number.

### Why This Matters in IT & Automation

- Parsing log files, CSV data, and API responses involves heavy string manipulation.
  - Validating and formatting output (reports, emails, dashboards) requires string methods.
  - Reading command-line arguments or config files returns strings that must be converted before math operations.
- 

## Lab Overview

|                |                                                                             |
|----------------|-----------------------------------------------------------------------------|
|                |                                                                             |
| Objectives     | Use string operators, indexing, slicing, and methods; convert between types |
| Estimated Time | 20–25 minutes                                                               |
| Prerequisites  | Lab 2.1 and 2.2 complete                                                    |

---

# Part A: String Operations

## Original Material

Strings are sequences of characters. Python provides powerful built-in tools for working with them.

## Code

```
first = "Hello"
last = "Python"

# Concatenation – joining strings
full = first + ", " + last + "!"
print(full)                # Hello, Python!

# Repetition
print("-" * 20)            # -----

# String length
print(len(full))           # 14

# Indexing – get one character (0-based)
print(full[0])              # H
print(full[-1])             # ! (last character)

# Slicing – get a substring
print(full[0:5])            # Hello
print(full[7:])             # Python!

# Common string methods
msg = " python is great "
print(msg.strip())          # "python is great"
print(msg.upper())          # " PYTHON IS GREAT "
print(msg.strip().capitalize()) # "Python is great"
print(msg.strip().replace("great", "awesome"))
```

## Expected Output

```
Hello, Python!
-----
14
H
!
Hello
Python!
python is great
  PYTHON IS GREAT
Python is great
python is awesome
```

## Code Execution

1. `first` and `last` are assigned string values.
2. `full = first + ", " + last + "!"` chains four strings together using `+` (concatenation) and binds the result to `full`.
3. `"-" * 20` uses the repetition operator to produce a string of 20 hyphens.

4. `len(full)` counts all characters in `full` — including the comma, space, and `!`.
5. `full[0]` accesses the character at index 0 (Python counts from zero). `full[-1]` accesses the last character using a negative index.
6. `full[0:5]` slices from index 0 up to (but **not including**) index 5.
7. `full[7:]` slices from index 7 to the end of the string (no upper bound = end).
8. String methods (`strip`, `upper`, `capitalize`, `replace`) return **new strings** — the original `msg` is never modified.

## Step-by-Step Explanation

| Expression                                | Result              | Notes                                                |
|-------------------------------------------|---------------------|------------------------------------------------------|
| <code>first + ", " + last + "!"</code>    | "Hello, Python!"    | Concatenation with +                                 |
| <code>"-" * 20</code>                     | "-----"             | Repetition with *                                    |
| <code>len(full)</code>                    | 14                  | Counts every character including punctuation/spaces  |
| <code>full[0]</code>                      | "H"                 | Index 0 = first character                            |
| <code>full[-1]</code>                     | "!"                 | Index -1 = last character                            |
| <code>full[0:5]</code>                    | "Hello"             | Slice: positions 0,1,2,3,4 (5 is excluded)           |
| <code>full[7:]</code>                     | "Python!"           | Slice from 7 to end                                  |
| <code>msg.strip()</code>                  | "python is great"   | Removes leading/trailing whitespace                  |
| <code>msg.upper()</code>                  | " PYTHON IS GREAT " | Converts all letters to uppercase (spaces preserved) |
| <code>msg.strip().capitalize()</code>     | "Python is great"   | First letter uppercase, rest lowercase               |
| <code>.replace("great", "awesome")</code> | "python is awesome" | Replaces every occurrence of the substring           |

## Understanding String Indexing

```
String:  H e l l o ,   P y t h o n !
Index:   0 1 2 3 4 5 6 7 8 9 10 11 12 13
Neg:    -14 -13 -12 -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1
```

Slicing syntax: `string[start:stop:step]`

- `stop` is **exclusive** (the character at `stop` is NOT included)
- Omitting `start` defaults to 0; omitting `stop` defaults to end of string

- `step` defaults to 1; `step = -1` reverses the string (`string[::-1]`)
- 

## Part B: Type Conversion

### Original Material

Python does not automatically mix types in operations. Use conversion functions when needed.

### Code

```
# Converting between types
num_str = "42"
num_int = int(num_str)      # string -> integer
num_float = float(num_str)  # string -> float

print(num_int + 8)          # 50
print(num_float + 0.5)      # 42.5

# Converting numbers to strings
score = 95
message = "Your score is: " + str(score)
print(message)              # Your score is: 95

# Getting user input (input() always returns a string)
# age_input = input("Enter your age: ")
# age = int(age_input)
# print("Next year you will be:", age + 1)
```

### Expected Output

```
50
42.5
Your score is: 95
```

### Code Execution

1. `num_str = "42"` — the value 42 is stored as a **string**, not a number.
2. `int(num_str)` parses the string and returns the integer 42. Now arithmetic is possible.
3. `float(num_str)` parses the string and returns 42.0 (a float).
4. `num_int + 8` → 50; `num_float + 0.5` → 42.5 — both work because types now match.
5. `str(score)` converts integer 95 to the string "95", allowing concatenation with "Your score is: ".
6. The commented `input()` block demonstrates the common pattern: read → convert → compute.

## Type Conversion Functions

| Function              | Converts To         | Example                    | Result |
|-----------------------|---------------------|----------------------------|--------|
| <code>int(x)</code>   | Integer             | <code>int("42")</code>     | 42     |
| <code>int(x)</code>   | Integer (truncates) | <code>int(3.9)</code>      | 3      |
| <code>float(x)</code> | Float               | <code>float("3.14")</code> | 3.14   |
| <code>str(x)</code>   | String              | <code>str(100)</code>      | "100"  |
| <code>bool(x)</code>  | Boolean             | <code>bool(0)</code>       | False  |

## Key Concepts

| Concept              | Definition                                                                                              |
|----------------------|---------------------------------------------------------------------------------------------------------|
| String concatenation | Joining strings using <code>+</code>                                                                    |
| String repetition    | Repeating a string using <code>*</code>                                                                 |
| Indexing             | Accessing a single character by its position (0-based)                                                  |
| Negative index       | Counting from the end: <code>-1</code> = last character                                                 |
| Slicing              | Extracting a substring:<br><code>s[start:stop:step]</code>                                              |
| String method        | A function belonging to a string object, called with dot notation                                       |
| Type conversion      | Explicitly changing a value's type using <code>int()</code> , <code>float()</code> , <code>str()</code> |
| <code>input()</code> | Always returns a <code>str</code> ; must be converted before numeric operations                         |

## Common Mistakes

**WRONG** — Concatenating a string and a number directly

```
score = 95
print("Score: " + score)    # TypeError: can only concatenate str (not "int") to str
```

**CORRECT** — Convert the number first

```
print("Score: " + str(score))    # Score: 95
# Or use an f-string (cleaner):
print(f"Score: {score}")        # Score: 95
```

**WRONG** — Forgetting that the slice `stop` is exclusive

```
word = "Hello"
print(word[0:5])    # "Hello" – correct, 5 IS excluded but string ends at 4
print(word[1:3])    # "el" – not "ell", position 3 is excluded
```

**CORRECT** — The rule: `stop` index is **never** included in the slice.

---

**WRONG** — Calling `int()` on a non-numeric string

```
int("hello")        # ValueError: invalid literal for int() with base 10: 'hello'
int("3.14")          # ValueError: int() can't convert a float string directly
```

**CORRECT** — Convert in stages if needed

```
int(float("3.14"))   # 3 (first to float, then to int)
```

---

**WRONG** — Thinking string methods modify the original string

```
msg = " hello "
msg.strip()           # Does nothing visible – the result is discarded!
print(msg)            # Still " hello "
```

**CORRECT** — Assign the result back

```
msg = msg.strip()     # Reassign to capture the new string
print(msg)            # "hello"
```

---

## Cross References

- **Previous concept:** [Lab\_2\_2\_Operators.md] — Arithmetic operators
- **Practice:** [Practice\_Problems.md] — Problem 2 (string analysis), Problem 1 (type conversion in formula)
- **Challenge:** [Challenge\_Lab.md] — Unit Converter (uses all three concepts)
- **Quick reference:** [Revision\_Sheet.md] — Important Functions Table, Key Syntax